Selected Topics in Visual Recognition using Deep Learning

HW1: Image classification

CHENG-CHIH, TSAI

112550183

(A) Introduction to the task and core idea of your method

This homework focuses on Image Classification problem. The primary task is to classify RGB images into 100 distinct object categories and should utilize ResNet-based architecture.

The core idea of my method focuses on solving the overfitting problem.

(B) Method: Describe your data preprocessing procedure, model architecture, hyperparameter settings, and any modifications or adjustments you made.

- Data Pre-processing: To enhance model robustness and prevent overfitting, applied Random Resized Crop, Horizontal Flip, and Color Jittering as data augmentation techniques. The images were also normalized using ImageNet standard means and standard deviation.

```python
# ========================================
# Data Augmentation
# ========================================
def set_transform(imgResize):
    MEAN = [0.485, 0.456, 0.406]
    STD = [0.229, 0.224, 0.225]

    trainTransform = transforms.Compose([
        transforms.RandomResizedCrop(imgResize[0], scale=(0.7, 1.0)),
        transforms.RandomHorizontalFlip(),
        transforms.TrivialAugmentWide(),
        transforms.ToTensor(),
        transforms.Normalize(mean=MEAN, std=STD),
        transforms.RandomErasing(p=0.2),
    ])

    valTransform = transforms.Compose([
        transforms.Resize(int(imgResize[0] * 1.15)),
        transforms.CenterCrop(imgResize[0]),
        transforms.ToTensor(),
        transforms.Normalize(mean=MEAN, std=STD),
    ])

    logger.debug(f"[set_transform] imgResize={imgResize}")
    return trainTransform, valTransform
```

- Model Architecture: Employed ResNet50 as the backbone. The fully connected layer was modified to use StarHead.

```python
class StarHead(nn.Module):
    def __init__(self, in_features, num_classes, dropout_rate=0.5):
        super(StarHead, self).__init__()

        self.fc1 = nn.Linear(in_features, in_features)
        self.fc2 = nn.Linear(in_features, in_features)

        # activate function
        self.act = nn.GELU()

        self.dropout = nn.Dropout(p=dropout_rate)
        self.classifier = nn.Linear(in_features, num_classes)

    def forward(self, x):
        # 1. Star Operation
        x1 = self.fc1(x)
        x2 = self.fc2(x)
        x_star = x1 * x2

        # 2. activate func and Dropout
        x_out = self.act(x_star)
        x_out = self.dropout(x_out)

        return self.classifier(x_out)
```

```python
def build_model_StarHead(model, numClasses=100, dropoutRate=0.5):
    inFeatures = model.fc.in_features

    model.fc = StarHead(in_features=inFeatures,
                        num_classes=numClasses,
                        dropout_rate=dropoutRate)

    logger.debug("[build_model] fc layer use StarHead: "
                 f"in_features={inFeatures}, num_classes={numClasses}")
    return model
```

The total number of parameters:

```
[INFO] 17:35:46 - ==================================================
[INFO] 17:35:46 - DLCV HW1 - Inference (Single Model)
[INFO] 17:35:46 - Using device: cuda
[INFO] 17:35:46 - ==================================================
[INFO] 17:35:46 - [Config] weight_path=./model_weight/resnet50_best_20260328_004701.pth
[INFO] 17:35:46 - [Config] img_resize=(448, 448), batch_size=64
[INFO] 17:35:46 - ========================================
[INFO] 17:35:46 - Total Params (M): 32.11 M
[INFO] 17:35:46 - ========================================
```

- Hyper:

```python
# ============================================================
# hyper setting
# ============================================================
batchSize = 32
numWorkers = 8
numEpochs = 100

learningRate = 1e-4
weightDecay = 5e-4
numClasses = 100
dropoutRate = 0.5

# ResNet50 (pretrained on ImageNet, ~25.6M params < 100M)
model = models.resnet50(weights="IMAGENET1K_V2")
use_StarHead = True

# Resize img for input
imgResize = (448, 448)
```

```
[INFO] 17:19:49 - [Seed] Set random seed = 60
[INFO] 17:19:49 - ==================================================
[INFO] 17:19:49 - DLCV HW1 - 100 img classification (ResNet)
[INFO] 17:19:49 - Using device: cuda
[INFO] 17:19:49 - ==================================================
[INFO] 17:19:49 - [Config] batch_size=32, epochs=100
[INFO] 17:19:49 - [Config] lr=0.0001, weight_decay=0.0005
[INFO] 17:19:49 - [Config] num_classes=100, img_resize=(448, 448)
[INFO] 17:19:49 - [Config] model=ResNet50
[INFO] 17:19:49 - [Training] Model moved to cuda
[INFO] 17:19:49 - [Training] Backbone lr=1e-05, Head lr=0.0001
[INFO] 17:19:49 - [Training] Optimizer=AdamW, weight_decay=0.0005
[INFO] 17:19:49 - [Training] Scheduler=CosineAnnealingLR, T_max=100
[INFO] 17:19:49 - [Data] load 20724 train data, total 100 classes
[INFO] 17:19:49 - [Data] load 300 valid data, total 100 classes
```

```python
# label smoothing
label_smoothing = 0.1
criterion = nn.CrossEntropyLoss(label_smoothing=label_smoothing)
```

(C) Results: your findings / model performance (e.g., training curve, confusion matrix, etc.)

(D) References: Your method references
(paper / Github sources, must include if you use any).

《TrivialAugment: Tuning-free Yet State-of-the-Art Data Augmentation》

《Rewrite the Stars》: github

(E) Additional experiments to explore better performance

- Simply tuning the hyper-parameters doesn't count
  (e.g., batch-size, LR, different optimizers).
- Hint: Try to add/remove some layers, use different loss functions, etc.
- Should include
  I. My hypothesis (why you do this),
     To solve the overfitting problem, focus on data processing.
     - Data Augmentation: Hand-crafted augmentations were replaced with TrivialAugmentWide.
     - High-Resolution Input: The standard input resolution was increased from 224×224 to 448×448
     - StarHead: Not only does it have low parameter, but also significantly improves generalization ability
  II. How this may (or may not) work, and
     - This parameter-free automated augmentation strategy increases data diversity without completely destroying semantic structures.
     - Upscaling the resolution effectively reduced the receptive field relative to the image size, allowing the ResNet backbone to capture much finer local details.
     - StarHead forces the model to understand more complex nonlinear relationships through implicit high-dimensional feature interactions such as x1 * x2.
  III. The experiment results and their implications.
     Without anything:

```
[INFO] 12:36:55 - Epoch [100/100] lr=0.000000
[INFO] 12:36:55 -    Train Loss: 0.8622 | Train Acc: 0.98900
[INFO] 12:36:55 -    Val   Loss: 1.2270 | Val   Acc: 0.88333
[INFO] 12:36:55 - ----------------------------------------------------
[INFO] 12:36:55 - [Training] Train finish, Best Val Acc: 0.88000 Best Val Loss: 1.22090
wandb:
wandb: Run history:
wandb:          Epoch
wandb:  Learning_Rate
wandb: Train/Accuracy
wandb:     Train/Loss
wandb:   Val/Accuracy
wandb:       Val/Loss
wandb:
wandb: Run summary:
wandb:          Epoch 100
wandb:  Learning_Rate 0
wandb: Train/Accuracy 0.989
wandb:     Train/Loss 0.8622
wandb:   Val/Accuracy 0.88333
wandb:       Val/Loss 1.22699
```

654365	prediction_20260328_171832_20260328.zip	2026-03-28 17:38	Finished	0.93

Add everything:

```
[INFO] 04:10:07 - Epoch [100/100] lr=0.000000
[INFO] 04:10:07 -    Train Loss: 0.8371 | Train Acc: 0.99334
[INFO] 04:10:07 -    Val   Loss: 1.0955 | Val   Acc: 0.92333
[INFO] 04:10:07 - ----------------------------------------------------
[INFO] 04:10:07 - [Training] Train finish, Best Val Acc: 0.91667 Best Val Loss: 1.09366
wandb:
wandb: Run history:
wandb:          Epoch
wandb:  Learning_Rate
wandb: Train/Accuracy
wandb:     Train/Loss
wandb:   Val/Accuracy
wandb:       Val/Loss
wandb:
wandb: Run summary:
wandb:          Epoch 100
wandb:  Learning_Rate 0
wandb: Train/Accuracy 0.99334
wandb:     Train/Loss 0.8371
wandb:   Val/Accuracy 0.92333
wandb:       Val/Loss 1.09551
```

654369	prediction_20260328_111258_20260328.zip	2026-03-28 17:39	Finished	0.96

(F) My Github link:

https://github.com/TsaiPercy/DLCV_2026_spring

(G) Code Reliability

```
/BCI_/DLCV/DLCV_2026_spring/hw1$ flake8 .
/BCI_/DLCV/DLCV_2026_spring/hw1$
```